

A flying capacitor three-cell voltage converter for studies in the control of switched systems

Daniel Vieira, Roberto Kawakami Harrop Galvão*, José Roberto Colombo Júnior

List all authors. Please mark the corresponding author with ()*

Instituto Tecnológico de Aeronáutica, Electronics Engineering Division, 12228-900, São José dos Campos, SP - Brazil

Please include the full address of each author institution

kawakami@ita.br

Institutional email address preferred. If you have a Twitter handle, please add it here 'twitter:'

Abstract

The challenges associated with implementing controllers for switched systems often arise from the high cost and complexity of the required hardware. This paper addresses that barrier by presenting a low-cost circuit for a three-cell voltage converter specifically designed for the experimental study of switched-mode control. The proposed solution uses standard, easily accessible electronic components, making it an ideal platform for educational and research purposes. At the heart of the control architecture is an ESP32 microcontroller, which provides the necessary real-time capabilities to generate precise signals for the switched actuators. This approach aims to provide a practical and easy-to-implement solution for developing and testing control algorithms, thus lowering the entry barrier for students and researchers in the field. This work contributes a valuable tool that bridges the gap between the theoretical aspects of switched systems and their accessible, hands-on application.

Keywords

Add at least 3 keywords and a maximum of 6 keywords.

Specifications table

Please replace the italicized instructions in the right column of the table with the relevant information about your hardware.

Hardware name	<i>Python-controlled three-cell converter</i>
Subject area	Educational tools and open source alternatives to existing infrastructure
Hardware type	Electrical engineering and computer science
Closest commercial analog	<i>Please specify the closest commercial analog to the submitted hardware. What would this hardware replace? If no commercial analog exists, state “No commercial analog is available.” Algo que tenha sido usado nos artigos da literatura</i>
Open source license	<i>All designs must be submitted under an open source license (for more details see the Guide for Authors). Please specify the open source license you’ve selected here.</i>
Cost of hardware	<i>Insert an approximate cost only - we will ask you for a complete breakdown of costs in a later section (Bill of materials).</i>
Source file repository	<i>If you’ve uploaded your source files to an approved repository (OSF, Mendeley Data or Zenodo) write the DOI URL here. For example: http://doi.org/10.17605/OSF.IO/WGK7Q</i>
OSHOWA certification UID (OPTIONAL)	<i>We encourage you to apply for a free OSHOWA Certification, which confirms your work is open-source compliant.</i> <i>If certification has been acquired, insert the OSHOWA UID here. For example: “CH000005”. In your OSHOWA certification project description, include a link to your HardwareX publication and the tag #HX.</i> <i>If you haven’t acquired certification, please delete this row of the specifications table.</i>

1. Hardware in context

During our research, we observed that most of the work in the field of switched-mode control for multi-cell converters was purely digital, relying heavily on simulation. The limited experimental work available often outsourced the design and manufacturing of the physical circuit to external companies, as documented in the work of Solano and Saenz. Our hardware aims to fill this gap by providing an easily accessible and low-cost solution for implementing and studying these complex control strategies. By using standard components and an embedded controller, our circuit provides an effective platform for hands-on research, reducing the barriers to entry for researchers and students.

2. Hardware description

Describe your hardware, highlighting the customization rather than the steps involved in the procedure. Explain how it differs from other hardware and the advantages it offers over pre-existing methods. For example, how does this hardware compare to other hardware in terms of cost or ease of use, or how can it be used to develop further designs in a particular area?

Add 3-5 bullet points which broadly explain to other researchers - inside or outside of the original user community - how the hardware could help them, with either standard or novel laboratory tasks.

Figuras:

- Tela do osciloscópio: Para um dado valor de α –; Três sinais de comando, v_{c1} , v_2 , v_r . Apresentar v_{c1} , v_2 , v_r durante a transição entre dois valores diferentes de α .
- Figura com modulação triangular.
- Tela do software.

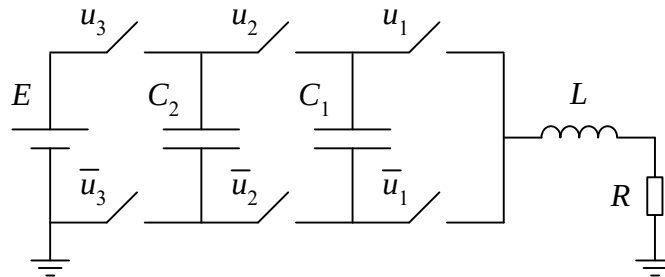


Figure 1: Three-cell Converter

Design files

Your design files should be editable - see [OSHWA's open source definition of 'Documentation'](#) for further details. You must then either:

- Upload your design files to one of the three approved online repositories - [Mendeley Data \(instructions\)](#), the [Open Science Framework \(instructions\)](#) or [Zenodo \(instructions\)](#). We recommend this option as the repositories support versioning of files.
- Upload your design files as supplementary materials (e.g., CAD files, videos...) to Hardware X's online editorial system when you submit your manuscript.
- Include your design files in the body of the manuscript (e.g., as figures).

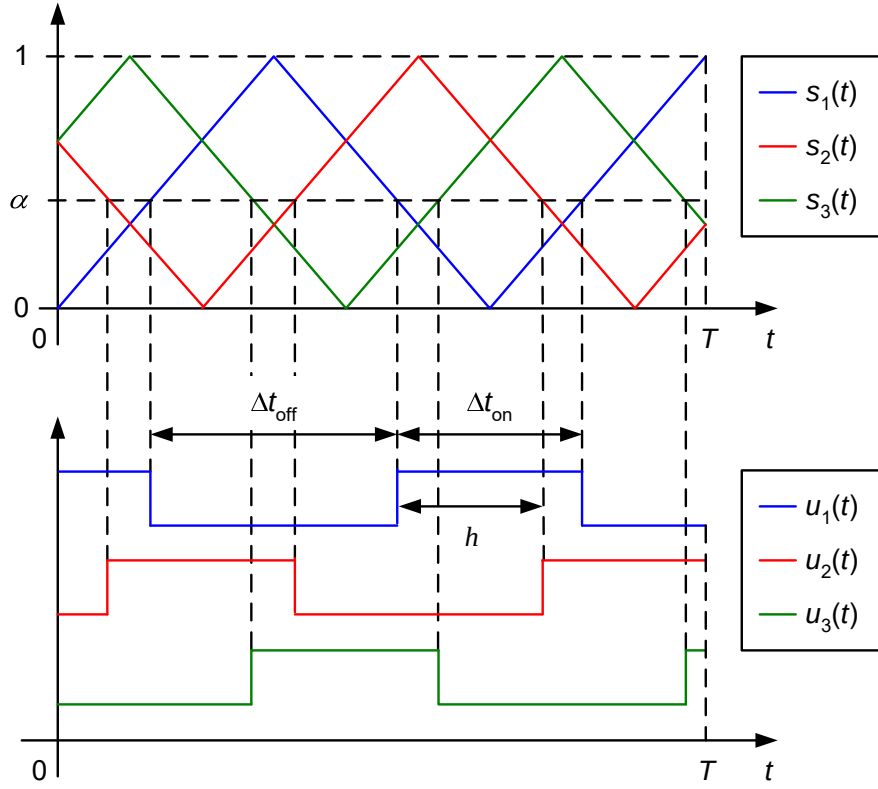


Figure 2: Circuit diagram.

CAD files: You are encouraged to use free and open source software packages for creating the files. For CAD files, [OpenSCAD](#), [FreeCAD](#), or [Blender](#) are encouraged, but, if these are not available, we accept source files from proprietary CAD packages, such as Autocad or Solidworks, and other drawing packages.

3D printing: Supplementary files that facilitate digital replication of the devices are encouraged; for example, STL files for 3D printing components. We recommend uploading CAD files to the [NIH 3D Print Exchange](#) as Custom Labware and then entering the link here.

Electronics: PCB layouts and other electronics design files can be uploaded to the [Open Hardware Repository](#) or other repositories or as supplementary materials.

Software and firmware: All software files used in the design and operation of the hardware should be included in the repository. Provide a description of the software and firmware and use extensive comments in the code.

3. Design files summary

Complete a separate row for each design file associated with your hardware (including the primary design files). Any empty rows should be deleted.

Design filename	File type	Open source license	Location of the file
For example: Design file 1	e.g. CAD file, figures, videos	All designs must be submitted under an open hardware license. Enter the corresponding open source license for the file.	Either enter the URL for the repository or the sentence: "Available with the article".
...	...	...	...
...	...	...	...

For each design file listed in the summary table above, include a short description of the file below (just one or

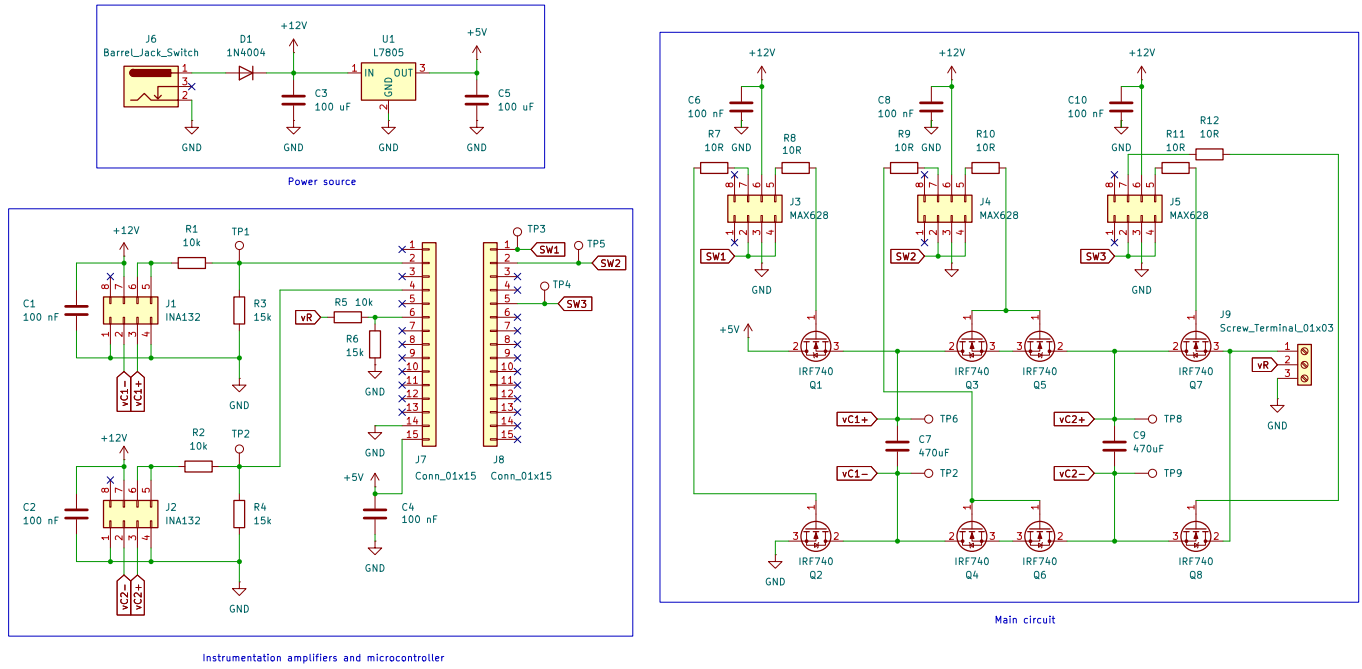


Figure 3: Schematics.

two sentences per design file).

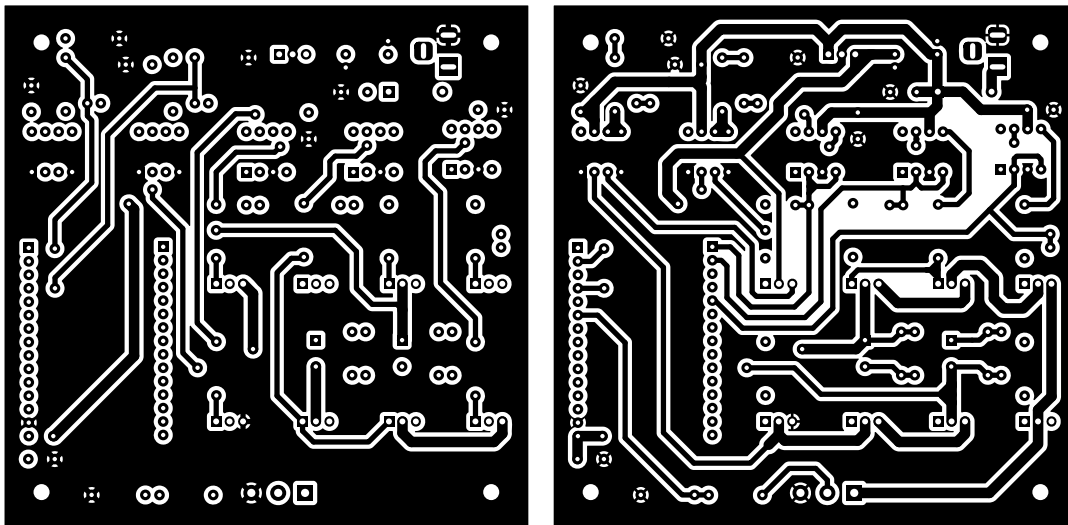


Figure 4: PCB layout: Front (left) and Back (right).

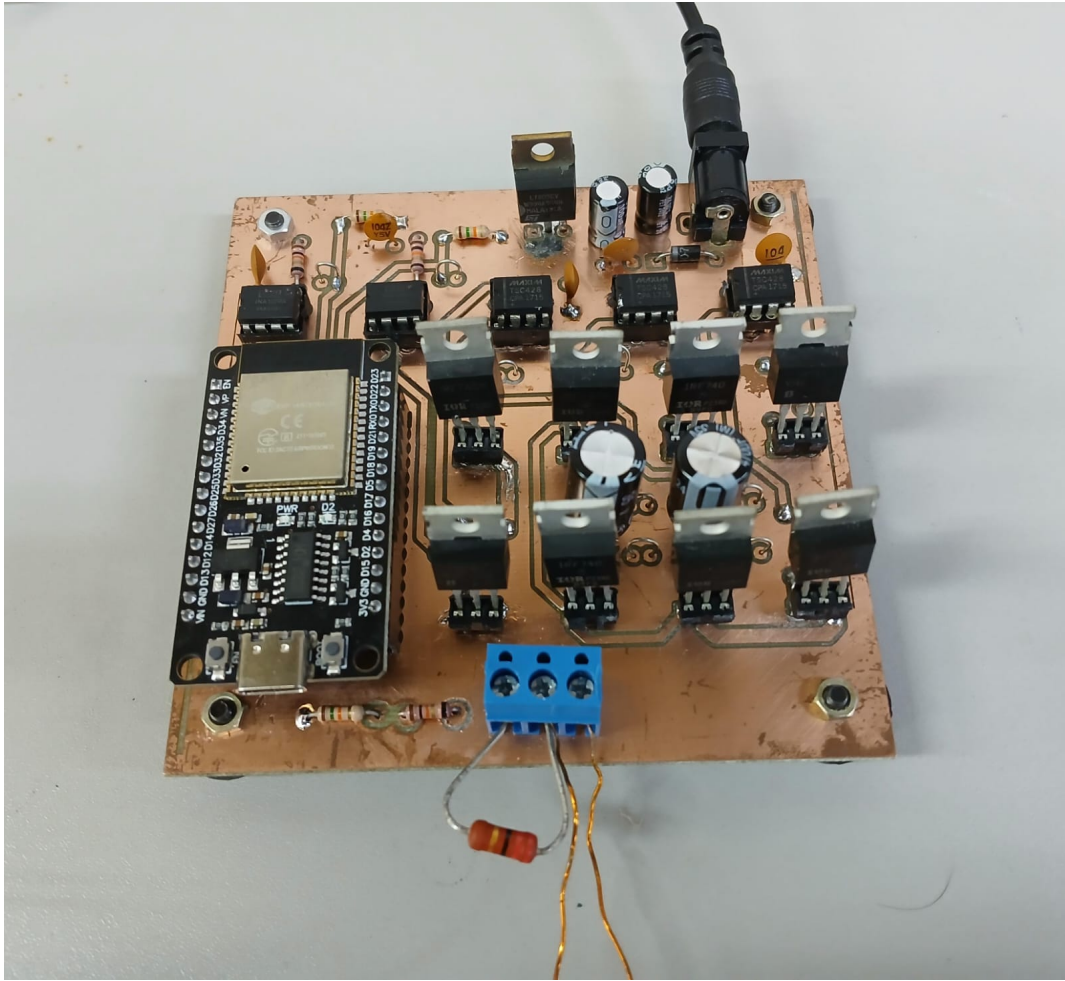


Figure 5: Photo of the device. The 10Ω resistors R1, R2, ..., R6 are soldered on the bottom.

4. Bill of materials summary

Designator	Component	Number	Cost per unit - currency	Total cost - currency	Source of materials	Material type
<i>If possible, use the same designator here as you use in the associated design file. If that's not possible, you will need to explain the relationship between the two.</i>	<i>Name of Component 1</i>	<i>Number of units</i>	<i>Cost per unit and the currency used</i>	<i>Total cost and the currency used</i>	<i>If possible, include a direct link to a webpage where the component can be purchased</i>	<i>Select from:</i> - Metal - Semiconductor - Ceramic - Polymer - Biomaterial - Organic - Inorganic - Composite - Nanomaterial - Semiconductor - Non-specific - Other
R1, R2, R3, R4, R5, R6	10 Ω Resistor	6	... 7	...	...	Electronic

You can use this space for any additional descriptions of the materials used.

5. Build instructions

Provide detailed, step-by-step construction instructions for the submitted hardware:

- *Include all necessary information for reproducing it.*
- *Explain and (when possible) characterize design decisions. Include any design alternatives you created.*
- *Use visual instructions such as schematics, images and videos.*
- *Clearly reference design files and component parts described in the **Design file summary** and the **Bill of materials summary**.*
- *Highlight any potential safety concerns.*

6. Software description

This chapter will elaborate on the software architecture. It is composed of a two-parts: a high-precision ESP32-based signal generator and a web-based remote control panel. This integration enables real-time configuration, control, and monitoring of the hardware directly from a web browser via Bluetooth Low Energy (BLE). We will deconstruct the architecture of both the ESP32 firmware and the React web application.

6.1 System Architecture

1. **The Embedded System (ESP32 Firmware):** A C++ application running on an ESP32 microcontroller. Its sole purpose is to generate precise digital signals on its GPIO pins and report back sensor data. It acts as a Bluetooth GATT server, waiting for commands and sending out data.
2. **The Web Control Interface (React App):** A web application built with React and TypeScript. It serves as the user's command center to define the signal patterns, send commands to the ESP32 and visualize incoming sensor data on a live chart. It acts as a Bluetooth GATT client, connecting to and controlling the ESP32.

The bridge between these two parts is the Web Bluetooth API, which allows the browser-based application to communicate wirelessly and directly with the ESP32 hardware.

6.2 Part 1: The Embedded System (ESP32 Firmware)

The firmware is built to be a robust, real-time signal generation engine leveraging features of the ESP32 and the FreeRTOS operating system to achieve high performance and reliability.

6.2.1 FreeRTOS: The Real-Time Engine

The ESP32 SDK is build on top of FreeRTOS, a real-time operating system designed for embedded devices. A RTOS guarantees that tasks can be executed with precise timing and this project uses FreeRTOS for:

- **Multitasking:** The firmware runs two primary tasks concurrently: 'signalTask' and 'bleTask'. The 'bleTask' handles all the non-time-critical Bluetooth communication, while the 'signalTask' focuses on managing the signal state machine
- **Task Pinning:** The ESP32 has a dual-core processor. To prevent the BLE stack from interfering with signal timing, both tasks are "pinned" to a specific CPU core using 'xTaskCreatePinnedToCore'. This prevents that a spike in Bluetooth activity on Core 1 will not delay the signal generation logic running alongside it.
- **Priority Management:** The 'signalTask' is assigned a high priority, ensuring it gets processor time when it needs, to enforce the real-time guarantees.

Here is how the tasks are created and pinned to CORE_1 and CORE_0

```
1 // from main.cpp
2
3 // Create BLE task on Core 1 with high stack size for BLE operations
4 xTaskCreatePinnedToCore(
5     bleTask,                // Task function
6     "BLE_Task",             // Task name
7     8192,                   // Stack size (bytes)
8     NULL,                   // Task parameter
9     2,                       // Priority
10    NULL,                    // Task handle
11    CORE_1                   // CPU core 1
12 );
```

```

13
14 // Create signal task on Core 1 with high priority for precise timing
15 xTaskCreatePinnedToCore(
16     signalTask,                // Task function
17     "Signal_Task",             // Task name
18     2048,                      // Stack size (bytes)
19     NULL,                      // Task parameter
20     configMAX_PRIORITIES - 2,  // High priority (second highest)
21     NULL,                      // Task handle
22     CORE_0                     // CPU core 0
23 );

```

Listing 1: Example of FreeRTOS Task Creation

6.2.2 ESP32 Hardware Timers

Software delays like ‘delay()’ are not capable of generating signals with microsecond accuracy. So, this system uses one of the ESP32’s built-in hardware timers.

- **How it Works:** The timer is a dedicated hardware that counts at a very high frequency. It is configured with a prescaler (TIMER_DIVIDER), which divides the CPU’s main clock (240MHz) to create a slower, more manageable timer clock (3MHz in this case).
- **Interrupt Service Routine (ISR):** The timer is set to trigger a hardware interrupt when its counter reaches a “alarm” value. This interrupt executes a function (Interrupt Service Routine). In this project, the ‘timer_group_isr_callback’ is the ISR. This function contains the code needed to switch the GPIO pins to their next state and set the alarm for the next interrupt.
- **IRAM Execution:** For maximum speed and to avoid delays from flash memory access, the ISR is loaded into the ESP32’s Instruction RAM (IRAM) using the IRAM_ATTR attribute.

```

1 // from main.cpp
2
3 bool IRAM_ATTR timer_group_isr_callback(void *args) {
4     // Re-enable timer alarm for next interrupt
5     timer_group_enable_alarm_in_isr(TIMER_GROUP, TIMER_IDX);
6
7     // Get references to the currently active signal set
8     std::vector<uint64_t>& time_vec = (active_set == ActiveSignalSet::SET_A) ?
9         time_vec_a : time_vec_b;
10    std::vector<uint64_t>& d4_vec = (active_set == ActiveSignalSet::SET_A) ?
11        d4_vec_a : d4_vec_b;
12    std::vector<uint64_t>& d5_vec = (active_set == ActiveSignalSet::SET_A) ?
13        d5_vec_a : d5_vec_b;
14    std::vector<uint64_t>& d6_vec = (active_set == ActiveSignalSet::SET_A) ?
15        d6_vec_a : d6_vec_b;
16
17    // Calculate next state position in signal pattern (circular buffer)
18    uint8_t next_state = (current_state + 1) % active_num_timings;
19
20    // Build pin state mask based on signal pattern
21    uint32_t pin_states =
22        (d4_vec[next_state] ? gpio_di4_mask : 0) |
23        (d5_vec[next_state] ? gpio_di5_mask : 0) |
24        (d6_vec[next_state] ? gpio_di6_mask : 0);
25
26    // Apply pin states using direct GPIO register manipulation for speed

```

```

23     GPIO.out_wits = pin_states;    // Set high pins
24     GPIO.out_witc = ~pin_states & (gpio_di4_mask | gpio_di5_mask | gpio_di6_mask);
        // Clear low pins
25
26     // Update current state and set next timer alarm
27     current_state = next_state;
28     timer_set_alarm_value(TIMER_GROUP, TIMER_IDX, time_vec[current_state]);
29
30     return true;
31 }

```

Listing 2: Core ISR logic

6.2.3 Bluetooth Low Energy (BLE)

Bluetooth Low Energy is a wireless communication protocol designed for sending small amounts of data with very low power consumption. It operates on a server-client model known as GATT (Generic Attribute Profile).

- **GAT Server (The ESP32):** The ESP32 is configured as a GAT Server. It creates and advertises a Service (identified by SERVICE_UUID).
- **Characteristics (The Data Points):** Inside the service, it defines a Characteristic (identified by CHARACTERISTIC_UUID). This is the actual endpoint for communication. The firmware configures this characteristic to be writable and to support notifications (to send data back to the client).
- **NimBLE Library:** The project uses the NimBLE-Arduino library, a highly efficient and memory-optimized BLE stack for the ESP32.

The BLE service and characteristic are defined and created in main.cpp

```

1  // from main.cpp
2
3  // BLE Service and Characteristic UUIDs for remote control
4  #define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
5  #define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
6
7  // ... inside bleTask() ...
8  NimBLEDevice::init("ESP32_Signal_Controller");
9  NimBLEServer *pServer = NimBLEDevice::createServer();
10 // ...
11 NimBLEService *pService = pServer->createService(SERVICE_UUID);
12 NimBLECharacteristic *pCharacteristic = pService->createCharacteristic(
13     CHARACTERISTIC_UUID,
14     NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::NOTIFY
15 );
16 pCharacteristic->setCallbacks(new CharacteristicCallbacks());
17 }

```

Listing 3: BLE service

6.3 Part 2: The Web Control Interface

This is the user interface of the system for controlling the ESP32. It is build with web technologies to run in any browser that supports Web Bluetooth (like Google Chrome)

6.3.1 Core Capabilities

- **React & TypeScript:** The application is build using React, a popular JavaScript library for building user interfaces, and TypeScript, which adds static typing to JavaScript. This combination allow a code that is both portable and maintainable.
- **Web Bluetooth API:** The Web Bluetooth API is a brawser standard that exposes Blueetooth functionality to JavaScript, allowing the application to

Scan for nearby BLE devices

Connect to a selected device (ESP32)

Discover its services and characteristics

Write data to characteristics (to send commands)

Subscribe to notifications from charateristics (to receive sensor data)

This eliminates the need for any native software, drivers, or browser plugins. The entire control system runs directly in the browser.

```
1 // from bluetooth.ts
2
3 export async function connect_device() {
4   try {
5     let navigatorObject: any = window.navigator;
6
7     // Open the device picker dialog, filtering for our specific service
8     g_bluetoothDevice = await navigatorObject.bluetooth.requestDevice({
9       filters: [{ services: [SERVICE_UUID] }],
10    });
11
12    // Connect to the device's GATT server
13    g_gattServer = await g_bluetoothDevice.gatt.connect();
14
15    // Get the service we're interested in
16    const service = await g_gattServer.getPrimaryService(SERVICE_UUID);
17
18    // Get the characteristic we'll use for reading/writing data
19    g_characteristic = await service.getCharacteristic(CHARACTERISTIC_UUID);
20
21  } catch (error: any) {
22    // Handle any errors during the connection process
23  }
24 }
```

Listing 4: Bluetooth connect_device

7. Operation instructions

Provide detailed, step-by-step instructions for the safe and proper operation of the hardware.

- *Use visual instructions, as necessary.*
- *Highlight any potential safety hazards.*

8. Validation and characterization

Demonstrate the operation of the hardware and characterize its performance for a specific scientific application.

- *Highlight a relevant use case.*
- *If possible, characterize performance of the hardware over operational parameters.*
- *Create a bulleted list describing the capabilities (and limitations) of the hardware. For example, load and operation time, spin speed, coefficient of variation, accuracy, precision, etc.*

Ethics statements

HardwareX has ethical guidelines that all authors must comply with. In addition, we ask you to complete the relevant statement(s) below. Please delete those which are not relevant for your work.

***If your work involved human subjects**, please include a statement here confirming that the relevant informed consent was obtained from those subjects:*

***If your work involved animal experiments**, please include a statement here confirming that those experiments complied with the ARRIVE guidelines and were carried out in accordance with the U.K. Animals (Scientific Procedures) Act, 1986 and associated guidelines; EU Directive 2010/63/EU for animal experiments; or the National Institutes of Health guide for the care and use of laboratory animals (NIH Publications No. 8023, revised 1978). Note, the sex of the animals must be indicated, and, where appropriate, the influence (or association) of sex on the results of the study:*

CRedit author statement

CRedit is an initiative that enables authors to share an accurate and detailed description of their diverse contributions to a published work.

*Example of a CRediT author statement: **Zhang San**: Conceptualization, Methodology, Software **Priya Singh**: Data curation, Writing- Original draft preparation. **Wang Wu**: Visualization, Investigation. **Jan Jansen**: Supervision. **Ajay Kumar**: Software, Validation.: **Sun Qi**: Writing- Reviewing and Editing.*

Please add a CRediT author statement for your data article here, using the [categories listed on this webpage](#).

Acknowledgements

Funding: This work was supported by Conselho Nacional de Desenvolvimento Científico e Tecnológico - CNPq, Brazil [grant number 305233/2022-0].

References

If relevant, you should include a reference to the original publication of the hardware you customized and a reference to the repository in which your design files are published. Other references can be included, as required; for example, references that put your device in context in the literature. For more information on the reference format in HardwareX please see the [Guide for Authors](#).

Author manuscript checklist

- Is the subject of the submission under an open source license? Are design files in the preferred format for making modifications as defined by the [Open Source Hardware definition](#)?
- Can the hardware be reproduced with the details provided in the submission?
- Are all relevant design files available on either the Mendeley Data, Open Science Framework, or Zenodo repository? Are they described in the Design Files Summary, and clearly documented? (E.g., descriptive file names, commented code, labeled images, etc.)
 - If in the Open Science Framework, has the repository been registered? [Instructions](#)
 - If in Zenodo, is the repository open access and published? [Instructions](#)
 - If in Mendeley Data, is the repository published or the sharable link included in the additional information you plan to submit? [Instructions](#)
- Are visual instructions used when necessary?
- Is the utility of the hardware to the scientific community explained clearly? Has a specific scientific application been demonstrated using the hardware?
- Is the performance of the hardware adequately demonstrated and characterized?
- Are all potential safety concerns addressed?
- For more information on the article template consult the [Guide to Authors](#).